

JavaScript Objects Review

Object Basics

- Definition:** An object is a data structure that is made up of properties. A property consists of a key and a value. To access data from an object you can use either dot notation or bracket notation.

```
1 const person = {
2   name: "Alice",
3   age: 30,
4   city: "New York"
5 };
6
7 console.log(person.name); // Alice
8 console.log(person["name"]); // Alice
```

"Alice"

"Alice"

To set a property of an existing object you can use either dot notation or bracket notation together with the assignment operator.

```
1 const person = {
2   name: "Alice",
3   age: 30
4 };
5
6 person.job = "Engineer"
7 person["hobby"] = "Knitting"
8 console.log(person); // {name: 'Alice', age: 30, job: 'Engineer', hobby
```

Removing Properties From an Object

- delete Operator:** This operator is used to remove a property from an object.

```
1 const person = {
2   name: "Alice",
3   age: 30,
4   job: "Engineer"
5 };
6
7 delete person.job;
```

undefined

Checking if an Object has a Property

- **`hasOwnProperty()`** Method: This method returns a boolean indicating whether the object has the specified property as its own property.

```
1 const person = {  
2   name: "Alice",  
3   age: 30  
4 };  
5  
6 console.log(person.hasOwnProperty("name")); // true  
7 console.log(person.hasOwnProperty("job")); // false
```

true

false

- **`Object.hasOwn()`** Method: This is the modern, recommended way to check if an object has a property as its own (not inherited). The syntax is `Object.hasOwn(object, propertyName)`. It returns `true` if the property exists on the object and `false` if it does not – regardless of the property's value. This makes it safer than `hasOwnProperty()` and more reliable than `if (obj.prop)` when values can be `0`, `false`, `null`, or `undefined`.

```
1 const person = {  
2   name: "Alice",  
3   age: 30  
4 };  
5  
6 console.log(Object.hasOwn(person, "name")); // true  
7 console.log(Object.hasOwn(person, "job")); // false
```

true

false

Here is a more detailed example showing why `Object.hasOwn()` is more reliable than checking the value directly:

```
1 const settings = {  
2   darkMode: false,  
3   fontSize: 0,
```

true

```
11
12 // Using if() directly is unsafe for falsy values!
13 if (settings.darkMode) {
14   console.log("Dark mode on"); // Does NOT print - misleading!
15 }
16
17 // Object.hasOwnProperty() is the safe way
18 if (Object.hasOwnProperty(settings, "darkMode")) {
19   console.log("darkMode exists, value is:", settings.darkMode); // darkM
20 }
```

```
false
```

- **in Operator:** This operator will return `true` if the property exists in the object.

```
1 const person = {
2   name: "Bob",
3   age: 25
4 };
5
6 console.log("name" in person); // true
```

```
true
```

Accessing Properties From Nested Objects

- **Accessing Data:** Accessing properties from nested objects involves using the dot notation or bracket notation, much like accessing properties from simple objects. However, you'll need to chain these accessors to drill down into the nested structure.

```
1 const person = {
2   name: "Alice",
3   age: 30,
4   contact: {
5     email: "alice@example.com",
6     phone: {
7       home: "123-456-7890",
8       work: "098-765-4321"
9     }
10  };
11 };
12
13 console.log(person.contact.phone.work); // "098-765-4321"
```

```
"098-765-4321"
```

Primitive and Non Primitive Data Types

- **Primitive Data Types:** These data types include numbers, strings, booleans, `null`, `undefined`, and symbols. These types are called "primitive" because they represent single values and are not objects. Primitive values are immutable, which means once they are

- **Definition:** Object methods are functions that are associated with an object. They are defined as properties of an object and can access and manipulate the object's data. The `this` keyword inside the method refers to the object itself, enabling access to its properties.

```
1 const person = {
2   name: "Bob",
3   age: 30,
4   sayHello: function() {
5     return "Hello, my name is " + this.name;
6   }
7 };
8
9 console.log(person.sayHello()); // "Hello, my name is Bob"
```

```
"Hello, my name is Bob"
```

Object Constructor

- **Definition:** In JavaScript, a constructor is a special type of function used to create and initialize objects. It is invoked with the `new` keyword and can initialize properties and methods on the newly created object. The `Object()` constructor creates a new empty object.

```
new Object()
```

Working with the Optional Chaining Operator (`?.`)

- **Definition:** This operator lets you safely access object properties or call methods without worrying whether they exist.

```
1 const user = {
2   name: "John",
3   profile: {
4     email: "john@example.com",
5     address: {
6       street: "123 Main St",
7       city: "Somewhere"
8     }
9   }
10 };
11
12 console.log(user.profile?.address?.street); // "123 Main St"
13 console.log(user.profile?.phone?.number);   // undefined
```

```
"123 Main St"
```

```
undefined
```

Object Destructuring

- **Definition:** Object destructuring allows you to extract values from objects and assign them to variables in a more concise and readable way.

```
1 const person = { name: "Alice", age: 30, city: "New York" };
2
3 const { name, age } = person;
```

```
"Alice"
```



Working with JSON

- **Definition:** JSON stands for JavaScript Object Notation. It is a lightweight, text-based data format that is commonly used to exchange data between a server and a web application.

```
{  
  "name": "Alice",  
  "age": 30,  
  "isStudent": false,  
  "list of courses": ["Mathematics", "Physics", "Computer Science"]  
}
```

- **JSON.stringify()**: This method is used to convert a JavaScript object into a JSON string. This is useful when you want to store or transmit data in a format that can be easily shared or transferred between systems.

```
1 const user = {  
2   name: "John",  
3   age: 30,  
4   isAdmin: true  
5 };  
6  
7 const jsonString = JSON.stringify(user);  
8 console.log(jsonString); // '{"name":"John","age":30,"isAdmin":true}'
```

```
"{"name":"John","age":30,"isAdmin":true}"
```



- **JSON.parse()**: This method converts a JSON string back into a JavaScript object. This is useful when you retrieve JSON data from a web server or localStorage and you need to manipulate the data in your application.

```
1 const jsonString = '{"name":"John","age":30,"isAdmin":true}';  
2 const userObject = JSON.parse(jsonString);  
3  
4 // result: { name: 'John', age: 30, isAdmin: true }  
5 console.log(userObject);
```

```
{ name: "John", age: 30, isAdmin: true }
```



Assignment

- ☐ Review the JavaScript Objects topics and concepts.

Submit

Ask for Help